Supabase Database Configuration, Schema, Test Log, and Fix Plan

Date: April 27, 2026
Requested freshness target: April 26, 2026
Project: PSScript
Hosted app: https://pstest.morloksmaze.com
Netlify site: `psscript` / `a6cb54b5-b3f7-4f01-a756-70b127f07e19`
Supabase project referenced by prior setup: `picxiqcekyfgjlrknfds`

Sources and Tools Used

- Supabase Postgres best-practices skill: `build-web-apps:supabase-postgres-best-practices`
- Netlify connector: confirmed project/deploy state
- Computer Use: confirmed production app browser state
- Live Supabase/Postgres read-only metadata probes through `DATABASE_URL`
- Existing Jest tests for DB runtime compatibility, AI analytics routes, and admin DB route behavior
- Supabase RLS docs: https://supabase.com/docs/guides/database/postgres/row-level-security
- Supabase query optimization docs: https://supabase.com/docs/guides/database/query-optimization
- Supabase connection pooling docs: https://supabase.com/docs/guides/database/connecting-to-postgres#connection-pooler
- Supabase pg_stat_statements docs: https://supabase.com/docs/guides/database/extensions/pg_stat_statements
- Supabase pgvector docs: https://supabase.com/docs/guides/database/extensions/pgvector
- Supabase Cron docs: https://supabase.com/docs/guides/cron
- PostgreSQL materialized view docs: https://www.postgresql.org/docs/current/rules-materializedviews.html

- Netlify Functions environment docs: https://docs.netlify.com/functions/environment-variables/

Netlify and Browser Confirmation

Netlify connector returned:

- Project name: `psscript`
- Site ID: `a6cb54b5-b3f7-4f01-a756-70b127f07e19`
- Primary URL: `https://pstest.morloksmaze.com`
- Current deploy: `69efe2d9eb552c3ef21786ee`
- Deploy state: `ready`
- Project access controls: no site password, no SSO team login requirement

Computer Use confirmed Chrome is on `pstest.morloksmaze.com/ai/agents`, the app is logged in, the Agent Orchestration chat page renders, and Voice Copilot is visible.

Configuration Inventory

Local env files found:

- `.env`
- `.env.example`
- `src/backend/.env`
- `src/backend/.env.example`
- `src/frontend/.env`
- `src/ai-service/.env.template`

Sensitive values were not printed. Relevant keys are present locally:

- `DATABASE_URL`
- `SUPABASE_URL`
- `SUPABASE_ANON_KEY`
- `SUPABASE_SERVICE_ROLE_KEY`
- `VITE_SUPABASE_URL` / `VITE_SUPABASE_ANON_KEY` in examples
- legacy `POSTGRES_*` local Docker variables, which are no longer part of the active production path
- `JWT_SECRET`

- `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`

Netlify Function DB config:

- `netlify/functions/_shared/db.ts` uses `pg.Pool`
- Default `DB_POOL_MAX` is `3`
- SSL defaults to enabled
- `DB_SSL_REJECT_UNAUTHORIZED` defaults to `true`
- Optional `DB_SSL_CA` / `DB_SSL_CA_PATH` are supported

Frontend Supabase auth config:

- `src/frontend/src/services/supabase.ts` requires `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`
- Hosted remote app fails closed when those are missing
- Localhost can use disabled auth only when explicitly configured

Live Database Connection Result

First strict TLS probe failed:

```
SELF_SIGNED_CERT_IN_CHAIN
```

Then a metadata-only fallback probe used encrypted SSL with certificate verification disabled. That fallback is acceptable for this review log only, not a recommended production posture.

Live DB metadata:

- Host kind: Supabase pooler
- Database: `postgres`
- Server version: PostgreSQL `17.6`
- Search path: `"$user", public, extensions`
- Extensions:
- `vector` `0.8.0` in `extensions`
- `pgcrypto` in `extensions`
- `pg_stat_statements` in `extensions`
- `uuid-ossp` in `extensions`

Hosted Tables Confirmed

Expected hosted app tables present:

Table	Exists	RLS
`app_profiles`	yes	enabled
`categories`	yes	enabled
`tags`	yes	enabled
`scripts`	yes	enabled
`script_versions`	yes	enabled
`script_tags`	yes	enabled
`script_analysis`	yes	enabled
`script_embeddings`	yes	enabled
`chat_history`	yes	enabled
`ai_metrics`	yes	enabled
`documentation_items`	yes	enabled
`hosted_artifacts`	yes	enabled
`script_file_hash_duplicate_audit`	yes	enabled
`schema_migrations`	yes	enabled
`analytics_events`	no	not implemented

Row counts from live read-only probe:

Table	Rows
`app_profiles`	3
`categories`	13
`tags`	3
`scripts`	7
`script_versions`	3
`script_tags`	0
`script_analysis`	5
`script_embeddings`	0
`chat_history`	0
`ai_metrics`	6
`documentation_items`	10
`hosted_artifacts`	0
`script_file_hash_duplicate_audit`	0

Migration State

The live `schema_migrations` table exists and contains these app migration names:

- `04_create_chat_history_table.sql`
- `20260412_create_ai_metrics_table.sql`
- `20260426_supabase_runtime_compatibility.sql`
- `20260412_fix_script_analysis_score_types.sql`
- `20260426_supabase_advisor_fixes.sql`
- `20260426_z_google_oauth_approval_gate.sql`

There are migrations in `supabase/migrations/` that are part of the hosted schema source:

- `20260424_hosted_schema.sql`
- `20260425_scripts_file_hash_uniqueness.sql`

- `20260425_user_management_schema_fixes.sql`
- `20260426_supabase_advisor_fixes.sql`
- `20260426_z_google_oauth_approval_gate.sql`

There are also local/legacy migrations in `src/db/migrations/`. The project currently has two schema tracks: hosted Supabase UUID/app_profiles and local legacy integer/users.

RLS and Policy Review

Good:

- All expected hosted public tables have RLS enabled.
- Owner/public script access is enforced in table policies.
- Disabled Google users are blocked with `current_app_profile_is_enabled()`.
- `auth.uid()` is mostly wrapped as `(select auth.uid())` in the newer OAuth approval migration, matching Supabase RLS performance recommendations.
- `current_app_profile_is_enabled()` is `SECURITY DEFINER` with explicit `search_path`.

Issues:

- `schema_migrations` has RLS enabled but no policies.
- `script_file_hash_duplicate_audit` has RLS enabled but no policies.
- Several policies call `public.current_app_profile_is_enabled()` directly instead of wrapping it as `(select public.current_app_profile_is_enabled())`. Supabase recommends wrapping stable helper functions in a subquery to avoid per-row execution.
- Policies are created for `{public}` role. The expressions protect user data, but using explicit `to authenticated` is clearer and safer for private user tables.
- `documentation_items` is readable by everyone. This is acceptable only if documentation content is intentionally public and never includes private/customer script details.

Index and Query Review

Good:

- Foreign key columns are mostly indexed.
- `script_analysis.script_id` is covered by its unique constraint.

- `scripts.user_id`, `scripts.category_id`, and file hash indexes exist.
- `script_embeddings` has a vector index.
- `chat_history` has an HNSW vector index.
- `pg_stat_statements` is enabled for query monitoring.

Issues:

- Missing FK index: `app_profiles.approved_by`.
- `scripts` lacks a composite or partial index for the production query shape:
- `WHERE user_id = $1 OR is_public = true ORDER BY updated_at DESC LIMIT ...`
- `scripts` keyword search uses `ILIKE` across `title`, `description`, and `content`; this will not scale without full-text or trigram indexes.
- `documentation_items` search uses `ILIKE` across `title` and `content`; live DB has only the vector index and primary key for this table.
- `ai_metrics` analytics filters by `created_at` plus `user_id`, but only has separate single-column indexes. A composite `(user_id, created_at desc)` index will match dashboard queries better.
- `script_embeddings` and `documentation_items` use IVFFlat indexes. With pgvector `0.8.0`, HNSW is usually the better default for incremental inserts. IVFFlat should be created after enough data exists or reindexed after backfill.

Read-only `EXPLAIN` summary:

- Script list/search and documentation search plans show sort nodes and are currently cheap only because data volume is tiny.
- AI metrics summary uses `idx_ai_metrics_created_at` plus filter on user/null; this should become a composite or partial-indexed query before metric volume grows.

Schema Drift

The hosted DB has:

- `script_embeddings.embedding_model`

The local base schema has:

- `script_embeddings.embedding_type`
- `script_embeddings.model_version`

Netlify code writes `embedding_model`, so hosted runtime is aligned with production, but the local base schema is not aligned with hosted. This is a source of future migration confusion.

Functional Data Gaps

- `script_embeddings` has `0` rows while `scripts` has `7` rows.
- `/api/scripts/:id/similar` in Netlify currently returns an empty hosted placeholder.
- Current hosted upload attempts to generate embeddings, but existing scripts are not backfilled.
- `chat_history` has `0` rows; hosted chat works but persistence is not currently populated through the Netlify chat path.
- `analytics_events` does not exist yet, which matches the separate analytics capture plan but is still a missing table for script/search product analytics.

Hosted API Probe

Unauthenticated hosted route probe:

Route	Status	Interpretation
`/api/health`	200	Hosted function and basic env checks respond
`/api/analytics/summary`	401	Correctly auth-gated
`/api/analytics/usage`	401	Correctly auth-gated
`/api/analytics/security`	401	Correctly auth-gated
`/api/analytics/categories`	401	Correctly auth-gated
`/api/analytics/ai/summary`	401	Correctly auth-gated

Test Log

Read-only/mocked backend DB-related tests:

```
PATH=/opt/homebrew/bin:$PATH npm test -- --runInBand \
  src/__tests__/database.runtime-compatibility.test.ts \
  src/routes/__tests__/analytics-ai.test.ts \
  src/routes/__tests__/admin-db.test.ts
```

Result:

- 3 test suites passed
- 16 tests passed

The opt-in live integration suite `RUN_DB_TESTS=true jest` was not run because it creates and deletes database rows. That is not appropriate against the hosted Supabase DB without explicit action-time confirmation.

CLI notes:

- `npx supabase --version` and `npx netlify env:list --plain` did not return output in this desktop session and were stopped.
- Netlify connector was used instead for project/deploy state.

Issues List

P1: Strict local TLS verification fails for `DATABASE_URL`

Evidence: Node `pg` read-only probe failed with `SELF_SIGNED_CERT_IN_CHAIN` when `rejectUnauthorized` was true.

Risk: Local and Netlify environments can drift. If Netlify does not have a valid CA path/value, server-side DB connections may fail under stricter SSL.

Fix:

- Add the Supabase pooler CA to `DB_SSL_CA` or `DB_SSL_CA_PATH`.
- Keep `DB_SSL_REJECT_UNAUTHORIZED=true` in production.
- Add a startup health check that fails if production connects only with certificate verification disabled.

P1: Search tables are not indexed for current search behavior

Evidence:

- `/scripts/search` uses `ILIKE` over title/description/content.
- `/documentation/search` uses `ILIKE` over title/content.
- Live `documentation_items` lacks GIN/trigram/full-text indexes.

Risk: Search becomes slow and expensive as scripts/docs grow.

Fix:

- Add `pg_trgm` or generated `tsvector` columns.
- Prefer full-text search for documents and scripts, with trigram for fuzzy fallback.
- Add date/order indexes for search result ordering.

P1: Script embeddings are empty and similar-script hosted route is still placeholder

Evidence:

- `scripts`: 7 rows.
- `script_embeddings`: 0 rows.
- Netlify `/scripts/:id/similar` returns an empty array placeholder.

Risk: Semantic search/similar script features appear present but do not work end-to-end.

Fix:

- Backfill embeddings for existing scripts.
- Add `ON CONFLICT (script_id) DO UPDATE` to embedding save.
- Implement hosted `/scripts/:id/similar` using pgvector with owner/public filtering.
- Add metrics for embedding generation failures.

P1: Schema drift between hosted and local script embedding columns

Evidence:

- Hosted DB has `embedding_model`.
- Local base schema has `embedding_type` and `model_version`.
- Netlify writes `embedding_model`.

Risk: New environments created from the wrong schema fail embedding writes or carry inconsistent metadata.

Fix:

- Standardize on one column name, preferably `embedding_model`.
- Add compatibility migration to local schema.
- Update docs and tests to expect the hosted shape.

P2: `app_profiles.approved_by` FK lacks an index

Evidence: Missing-FK-index heuristic found `app_profiles.approved_by`.

Risk: Admin approval queries/deletes can become slower at scale.

Fix:

```
create index if not exists idx_app_profiles_approved_by
  on public.app_profiles(approved_by)
  where approved_by is not null;
```

P2: RLS helper should be wrapped consistently

Evidence: Policies call `public.current_app_profile_is_enabled()` directly.

Risk: Potential per-row function execution overhead on large tables.

Fix:

- Replace direct calls in policies with `(select public.current_app_profile_is_enabled())`.
- Keep helper `SECURITY DEFINER` and explicit `search_path`.

P2: RLS-enabled maintenance tables have no policies

Evidence:

- `schema_migrations`
- `script_file_hash_duplicate_audit`

Risk: This is secure-by-default for direct clients but should be intentional and documented. It may surprise admin tooling if direct reads are expected.

Fix:

- Either document them as server-only/no direct access, or add admin-only SELECT policies backed by a security-definer admin check.

P2: `ai_metrics` needs dashboard-oriented composite indexes

Evidence: Analytics summary scans by date and filters by user/null.

Fix:

```
create index if not exists idx_ai_metrics_user_created
  on public.ai_metrics(user_id, created_at desc);

create index if not exists idx_ai_metrics_global_created
  on public.ai_metrics(created_at desc)
  where user_id is null;
```

P2: Product analytics table is missing

Evidence: `analytics_events` does not exist.

Risk: Script/search behavior cannot be measured beyond AI provider costs.

Fix: Implement `analytics_events` using the hosted Netlify + Supabase pattern described in current architecture docs. The older analytics capture plan is archived for history.

P3: Public read policies need explicit product decision

Evidence:

- `categories`: readable by everyone.
- `tags`: readable by everyone.
- `documentation_items`: readable by everyone.

Risk: Fine for public metadata, risky if documentation imports ever contain customer/private content.

Fix:

- Keep categories/tags public if intended.
- Move documentation to authenticated read if content may contain private imported docs.

Complete Project Fix Plan

Phase 1: Lock DB Connection and Schema Drift

1. Add Supabase CA config for strict TLS in local and Netlify.
2. Add a read-only health check that confirms strict TLS works.
3. Standardize `script_embeddings` metadata columns on `embedding_model`.
4. Add tests for hosted schema column names.

Verify:

- Strict TLS metadata probe succeeds.
- `/api/health` remains 200.
- Netlify deploy logs show no DB TLS warnings.
- `script_embeddings` insert works in a focused test.

Phase 2: Search and Index Fixes

1. Add `pg_trgm` or full-text search support.
2. Add script search indexes:

```
create extension if not exists pg_trgm with schema extensions;

create index if not exists idx_scripts_user_updated
  on public.scripts(user_id, updated_at desc);

create index if not exists idx_scripts_public_updated
  on public.scripts(updated_at desc)
  where is_public = true;

create index if not exists idx_scripts_title_trgm
  on public.scripts using gin (title extensions.gin_trgm_ops);

create index if not exists idx_scripts_description_trgm
  on public.scripts using gin (description extensions.gin_trgm_ops);
```

1. Add documentation search indexes:

```
create index if not exists idx_documentation_items_updated
  on public.documentation_items(updated_at desc);

create index if not exists idx_documentation_items_title_trgm
  on public.documentation_items using gin (title extensions.gin_trgm_ops);

create index if not exists idx_documentation_items_content_trgm
  on public.documentation_items using gin (content extensions.gin_trgm_ops
```

Verify:

- `EXPLAIN` for script and documentation searches uses indexes at realistic row counts.
- Existing search UI still returns the same response shape.

Phase 3: Semantic Search Completion

1. Backfill `script_embeddings` for all existing scripts.
2. Add `ON CONFLICT (script_id) DO UPDATE` to hosted embedding save.
3. Replace `/scripts/:id/similar` placeholder with pgvector query.
4. Prefer HNSW for live incremental vector indexes on pgvector 0.8.0.

Verify:

- `script_embeddings` count matches scripts with non-empty content.
- `/api/scripts/:id/similar` returns owner/public filtered results.
- A private script cannot discover another user's private script through similarity search.

Phase 4: RLS and Policy Cleanup

1. Add `idx_app_profiles_approved_by`.
2. Wrap `current_app_profile_is_enabled()` in `select` inside policies.
3. Convert private-table policies from implicit `{public}` to explicit `to authenticated`.
4. Decide and document whether `documentation_items` is public or authenticated.
5. Add admin-only policy or explicit server-only documentation for `schema_migrations` and duplicate audit table.

Verify:

- Owner can read own script and analysis.
- Disabled Google account cannot read/write app tables.
- Other user cannot read private scripts/analysis/embeddings.
- Public scripts remain readable if that product behavior is intended.

Phase 5: Analytics Schema

1. Add `analytics_events`.
2. Add `request_id` to `ai_metrics`.
3. Add Netlify `recordAnalyticsEvent`.
4. Capture script search, documentation search, AI search, result click, analysis, upload, download, and copy events without raw prompts/content.
5. Add daily rollups with Supabase Cron.

Verify:

- Analytics inserts are best-effort and never fail user requests.
- RLS blocks cross-user raw events.
- Search routes return `search_id`.
- Dashboards use aggregate queries.

Phase 6: Monitoring and Maintenance

1. Add `pg_stat_statements` review query to admin DB health output.
2. Add table/index bloat and dead tuple checks.
3. Add scheduled `analyze`/maintenance notes for large imports.
4. Add migration smoke test that applies hosted migrations to a clean database.

Verify:

- Admin DB panel shows slow/frequent query candidates.
- `schema_migrations` accurately reflects applied hosted migrations.
- Clean-database migration test passes.

Recommended Next Implementation Slice

Start with the smallest high-impact patch:

1. Add `idx_app_profiles_approved_by`.
2. Add script/doc search indexes.
3. Add `idx_ai_metrics_user_created`.
4. Add strict TLS CA configuration documentation and validation.
5. Add a read-only schema verification script/test that does not mutate hosted data.

Then move to the embedding backfill and `/scripts/:id/similar` implementation.

Final Status

The hosted Supabase database is connected, live, and broadly healthy: expected core tables exist, RLS is enabled, extensions are in place, and the production Netlify deploy is ready. The main issues are not basic setup failures; they are correctness/performance gaps that will matter as data grows: strict TLS trust configuration, search indexing, schema drift cleanup, empty embeddings/similarity placeholder, and missing product analytics events.

Generated 2026-04-29 14:08 UTC